

# Implementing Simple Subprograms

A subprogram is a block of code designed to perform a specific task. It helps divide a large program into smaller and manageable parts. They can take inputs, perform operations, and sometimes return results.

---

## Types of Subprograms

### 1. Function

- A function is a subprogram that returns a value.
  - Example: Addition of two numbers.
- 

### 2. Procedure

- A procedure performs an action but does not return a value.
  - Example: Printing a message.
- 

## Key Concepts in Implementing Simple Subprograms

### 1. Parameters and Arguments

- Parameters are variables defined in the subprogram.
- Arguments are actual values passed during function call.

**Example:**

In `multiply(4,5)`

- `a` and `b` are parameters
- `4` and `5` are arguments

---

## 2. Return Values

- Functions return a result after computation.
- The `return` statement sends the result back.

**Example:**

```
return x + y
```

---

## 3. Calling Subprograms

- A subprogram is executed by calling its name with arguments.

**Example:**

```
add_numbers(5,3)
```

---

## Testing of Subprograms

### 1. Unit Testing

- Tests individual subprograms separately.

### 2. Integration Testing

- Tests whether subprograms work together properly.
-

## Examples of Simple Subprograms

```
def add_numbers(x, y):
```

```
    return x + y
```

This function adds two numbers and returns the result.

---

## Advantages of Subprograms

- Reduces program complexity
- Reuses code easily
- Makes debugging simple
- Improves readability

## Nested Subprograms

A nested subprogram is a subprogram defined inside another subprogram. The inner subprogram can access variables and data of the outer subprogram.

---

### Working of Nested Subprograms

- One subprogram is created inside another subprogram.
  - The inner subprogram is usually used only within the outer subprogram.
  - It can access local variables of the outer subprogram.
- 

### Features of Nested Subprograms

1. **Better Program Organization** - Helps divide large programs into smaller units.
  2. **Data Hiding** - The inner subprogram is hidden from other parts of the program.
  3. **Variable Access** - Inner subprograms can use variables of outer subprograms.
  4. **Improved Readability** - Makes programs easier to understand and maintain.
-

## Example of Nested Subprogram

### Python Example

```
def outer():  
    message = "Hello"  
  
    def inner():  
        print(message)  
  
    inner()  
  
outer()
```

---

### Explanation of the program

- `outer()` is the outer subprogram.
- `inner()` is the nested subprogram inside `outer()`.
- The inner function accesses the variable `message`.

---

### Advantages of Nested Subprograms

- Reduces program complexity
- Improves modularity
- Increases code security
- Avoids unnecessary global variables

# Blocks and Dynamic Scoping

---

## Block

- A block is a user-defined local scope for variables in a program. Variables declared inside a block can be accessed only within that block.
  - They help avoid conflicts between variables with the same name.
- 

## Example of Block in C

```
{  
  int temp;  
  temp = list[upper];  
  list[upper] = list[lower];  
  list[lower] = temp;  
}
```

---

## Explanation

- Variable **temp** is created inside the block.
  - It exists only while the block is executing.
  - After execution, the variable is destroyed.
- 

## Implementing Blocks

Blocks can be implemented using two methods.

### Method 1: Treating Blocks as Parameter-less Subprograms

- Blocks are treated like subprograms without parameters.

## Method 2: Compile-Time Storage Allocation

- Maximum storage needed for a block is determined at compile time.
- 

## Dynamic Scoping

- Dynamic scoping means the value of a variable is determined by the runtime calling sequence instead of program structure.
  - When a variable is used, the program searches for its value in the sequence of function calls.
  - The first matching variable found is used.
- 

## Implementing Dynamic Scoping

There are two methods:

### 1. Deep Access

- Searches activation records in the dynamic chain.
- Search continues until a variable is found.

### 2. Shallow Access

- Variables are stored in a central table.
- Separate stack maintained for each variable name.

## Example of Dynamic Scoping

```
void sub3() {  
    int x, z;  
    x = u + v;  
}
```

## Explanation of the program

When `sub3()` executes, variables `u` and `v` are searched in the calling sequence:

`sub3 → sub2 → sub1 → main`

The first occurrence of the variable is used.

---

## Difference Between Static and Dynamic Scoping

| S.No | Static Scoping                 | Dynamic Scoping                 |
|------|--------------------------------|---------------------------------|
| 1    | Binding occurs at compile time | Binding occurs at run time      |
| 2    | Based on program structure     | Based on function call sequence |
| 3    | Faster execution               | Slower execution                |
| 4    | Easy to understand             | Difficult to trace              |